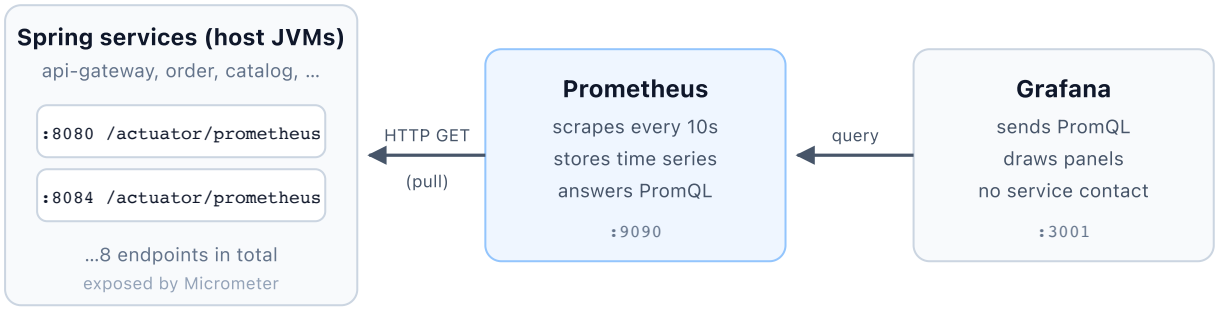
Metrics with Prometheus and Grafana

How the two tools actually work, and how they are wired into this project — eight Spring Boot services running as host JVMs, scraped by a Prometheus container, drawn by a provisioned Grafana dashboard.

Contents

1 The shape of the system

The single most important thing to understand is the direction of the arrows: **Prometheus pulls.** Your services do not send metrics anywhere. Each one simply publishes a plain-text page of current numbers at an HTTP endpoint and forgets about it. Prometheus visits that page on a schedule, parses it, and stores what it finds. Grafana never talks to your services at all — it only ever asks Prometheus.



Both arrows point left: nothing is ever pushed to Prometheus, and Grafana holds no data of its own — it re-queries on every refresh.

The pull model. A service that is down simply stops answering; Prometheus records that as `up=0` rather than silently losing data.

Two consequences fall straight out of this design, and they explain most of the behaviour you will see:

- **Prometheus must be able to reach your service, not the other way round.** That is why the network path matters so much in this project (§7) — the services run on the host while Prometheus runs in a container.
- **A stopped service exports nothing at all**, including its own labels. So you cannot ask "which services are down?" using a metric the service itself publishes. Prometheus therefore synthesises one metric per target on every scrape attempt: `up`, which is `1` on success and `0` on failure. It is the only metric guaranteed to exist for a dead target.

2 How Prometheus works

Prometheus is one process doing four jobs.

Job	What it means here
Service discovery	Deciding what to scrape. We use the simplest form, `static_configs` — a hand-written list of eight host:port targets. Larger setups discover targets from Kubernetes, Consul, or EC2 instead.
Scraping	Every `scrape_interval` (10s for us) it issues a plain HTTP GET to each target's `metrics_path` and parses the response.
Storage	Samples land in a local time-series database on disk — the `prometheus_data` volume in `docker-compose.yml`. Each sample is just (series, timestamp, float64).
Querying	It serves PromQL over an HTTP API. Grafana is simply a client of that API; the built-in UI at `:9090` is another.

> **Why pull rather than push?** Because the scrape itself is a health signal. With push, a silent service and a healthy-but-idle service look identical. With pull, failing to answer *is* the signal, and it is recorded as data you can alert on. It also means no service needs to know that a monitoring system exists.

3 What a metric actually looks like

There is no binary protocol and no SDK handshake. The endpoint returns text. This is genuine output from `order-service` in this project:

```
# HELP http_server_requests_seconds
# TYPE http_server_requests_seconds histogram
http_server_requests_seconds_count{application="order-service",method="GET",
  outcome="SUCCESS",status="200",uri="/actuator/health"} 9
http_server_requests_seconds_sum{application="order-service",method="GET",
  outcome="SUCCESS",status="200",uri="/actuator/health"} 0.095630291

jvm_memory_used_bytes{application="order-service",area="heap",id="G1 Eden Space"} 1.2582912E7
jvm_memory_used_bytes{application="order-service",area="heap",id="G1 Old Gen"} 5.9826984E7
```

Read one line as three parts: a **metric name**, a set of **labels** in braces, and a **value**. The name plus its exact label set defines a *time series*. Change one label value and it is a different series, tracked separately forever. That is why `jvm_memory_used_bytes` above is two series, not one — the `id` label differs.

Labels are the whole point. Because `application` is attached to every metric, one query can aggregate across all eight services or split them apart, with no per-service configuration anywhere.

The four metric types

Type	Behaviour	Example here
Counter	Only ever goes up (or resets to 0 on restart). You almost never read it directly — you take its `rate()`.	`http_server_requests_seconds_count`
Gauge	Goes up and down. Read as-is.	`jvm_memory_used_bytes`
Histogram	Counts observations into cumulative buckets (`le=` "less than or equal"), enabling percentiles computed server-side.	`http_server_requests_seconds_bucket`
Summary	Ships count and sum only (and Micrometer adds a max). Cheaper, but *cannot* produce percentiles after the fact.	What Spring publishes by default — see the warning below

> **A real bug this project hit.** Spring Boot publishes `http_server_requests_seconds` as a *summary* by default, so there are no `_bucket` series at all — and `histogram_quantile()` silently returns nothing. The dashboard's "Latency p95" panel read *No data* forever, with no error to explain why. The fix is one setting in `config-repo/application.yml`, which switches that timer to a real histogram:
>
> ```
> management.metrics.distribution.percentiles-histogram.http.server.requests: true
> ```

4 Where the numbers come from: Micrometer

No code in this project creates a metric by hand. The chain is:

1. **Micrometer** is a vendor-neutral metrics facade, already bundled with `spring-boot-starter-actuator`. Spring instruments HTTP handling, the JVM, connection pools, Kafka clients and more against it automatically.
2. Adding **`micrometer-registry-prometheus`** supplies a Prometheus-format *registry*. Its presence alone makes Actuator offer a `/actuator/prometheus` endpoint.
3. Exposing it over HTTP is opt-in, so the endpoint is listed explicitly: `management.endpoints.web.exposure.include: health,info,prometheus`.

One shared setting does the heavy lifting for a multi-service system — a common tag applied to every metric the process emits:

```
management.metrics.tags.application: ${spring.application.name}
```

Without it, eight services would publish eight identical-looking sets of `http_server_requests_seconds_count` and there would be no way to tell them apart after aggregation.

5 PromQL: turning counters into answers

A counter's raw value ("47,213 requests since this process started") is rarely interesting. PromQL's job is to turn accumulated totals into rates, percentiles and comparisons. Four expressions cover most of what the dashboard does.

```
# 1. Requests per second, per service, averaged over a 1-minute window.
sum by (application) (rate(http_server_requests_seconds_count[1m]))

# 2. Same, but only 5xx responses — status is just another label to filter on.
sum by (application) (rate(http_server_requests_seconds_count{status=~"5.."}[1m]))

# 3. 95th-percentile latency, computed from histogram buckets.
histogram_quantile(0.95, sum by (application, le) (rate(http_server_requests_seconds_bucket[5m])))

# 4. Which targets are alive right now.
up{job="spring-services"}
```

Three ideas explain all four:

- `rate(x[1m])` — per-second average increase of a counter over the last minute. It handles process restarts (counter resets) correctly, which naive subtraction does not.
- `sum by (application) (…)` — collapse every series down to one per service, discarding `uri`, `method`, `status` and the rest. Swap the label to slice differently; `sum by (uri)` would give per-endpoint traffic instead.
- `{status=~"5.."}` — label matching, here with a regex. This is why labels matter: filtering and grouping are the same mechanism.

> Note that `histogram_quantile` needs the `le` label preserved in the `sum by (application, le)` — the buckets *are* the distribution. Drop `le` and the function has nothing to interpolate.

6 How Grafana works

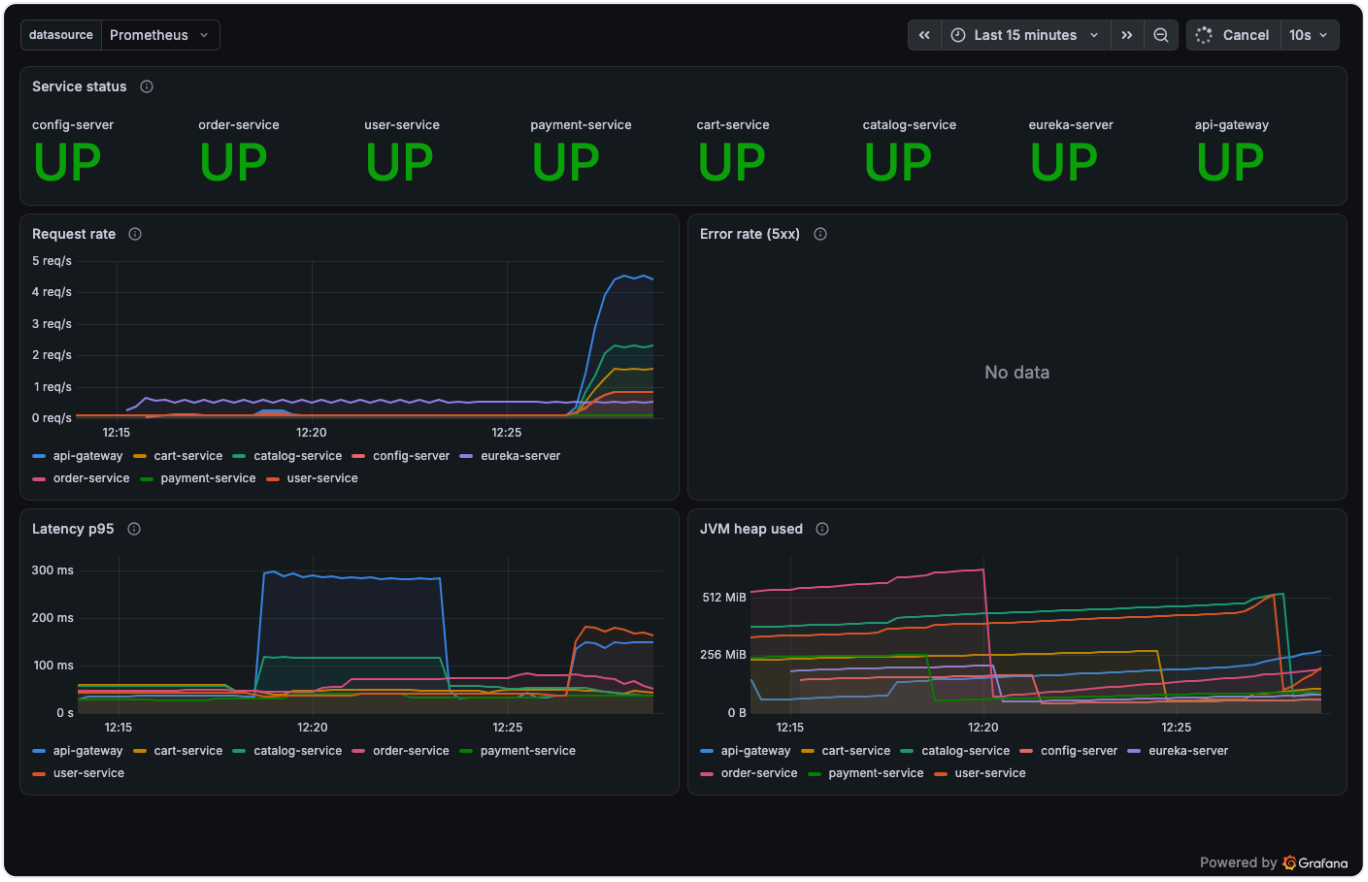
Grafana stores no metrics. Every panel is a saved query plus drawing instructions; on each refresh it re-runs the query against a **datasource** and renders whatever comes back. Three concepts:

Concept	Role
Datasource	A backend to query. Ours is Prometheus at `http://prometheus:9090` — a container name, because Grafana reaches it across the Compose network, not via localhost.

Panel	One query plus a visualisation (time series, stat, table…) and its formatting: units, thresholds, colours, legend.
Dashboard	A grid of panels sharing a time range and refresh interval. Stored as JSON — which is what makes it reviewable in git.

Provisioning: dashboards as code

Rather than clicking a dashboard together and hoping it is backed up, this project defines both the datasource and the dashboard as files that are mounted into the container. Grafana reads them at startup, so a fresh `docker compose up` always produces the same dashboard, and changes go through code review like anything else.



The provisioned dashboard with all eight services running and traffic flowing through the gateway. Each service keeps its own colour across every panel — that is the pinned-palette rule below, not Grafana's default ordering. "Error rate" reads *No data* because no 5xx occurred: an empty result is not a broken panel. The step in "JVM heap used" is a service restart.

Two deliberate choices in this dashboard

- **Status is never carried by colour alone.** The tiles print `UP` / `DOWN` as text next to the colour, so the panel still reads correctly in greyscale or for a colour-blind viewer.
- **Colours are pinned per service name, not by query order.** Grafana's default palette assigns colours by the order series arrive, so `order-service` could be blue on one panel and orange on the next, or change colour when another service goes down. Each of the eight services is instead given a fixed hue in the dashboard JSON, so a colour always means the same service.

7 This project's wiring, file by file

File	What it contributes
`services/*/pom.xml`	`micrometer-registry-prometheus` on all eight services — this is what creates the endpoint.
`config-repo/application.yml`	Exposes the `prometheus` endpoint, adds the `application` tag to every metric, and enables latency histogram buckets. Served to the six business services by config-server.
`services/eureka-server/…/application.yml` `services/config-server/…/application.yml`	The same actuator settings, repeated locally — these two cannot read config-repo. See below.
`monitoring/prometheus/prometheus.yml`	The eight scrape targets and the 10-second interval.
`monitoring/grafana/provisioning/`	Datasource and dashboard-provider definitions read at Grafana startup.
`monitoring/grafana/dashboards/`	The dashboard JSON itself.
`docker-compose.yml`	The `prometheus` and `grafana` containers, their mounts and ports.

The two services that cannot read the shared config

Six of the eight services fetch `config-repo/application.yml` from config-server at startup, so the actuator settings reach them for free. **Two cannot**, by definition:

- **config-server** is the thing serving that file — it cannot import from itself.
- **eureka-server** is the registry that config-server registers *with*, so it comes up first, before any config is available.

Both therefore bootstrap from their own `application.yml` only, and the shared metrics settings have to be repeated there. Miss this and the two failures look unrelated, though the cause is identical:

Service	Symptom before the fix
`eureka-server`	A plain **404** — the endpoint was never exposed.
`config-server`	`unsupported Content-Type "application/json"` — far stranger, and explained below.

> **Why config-server answered with JSON.** Spring Cloud Config Server maps `/{application}/{profile}` at the root. When `/actuator/prometheus` was *not* an exposed endpoint, nothing claimed that path, so the config handler took it — reading it as a request for the application named `actuator`, profile `prometheus` — and returned a perfectly valid config document. Prometheus then rejected it for having the wrong content type. Note that `/actuator/health` was never affected: that

> endpoint *does* exist, so the actuator mapping wins the path. Exposing `prometheus` is the whole fix; no path prefix or separate management port is needed.

The one genuinely awkward part: crossing the container boundary

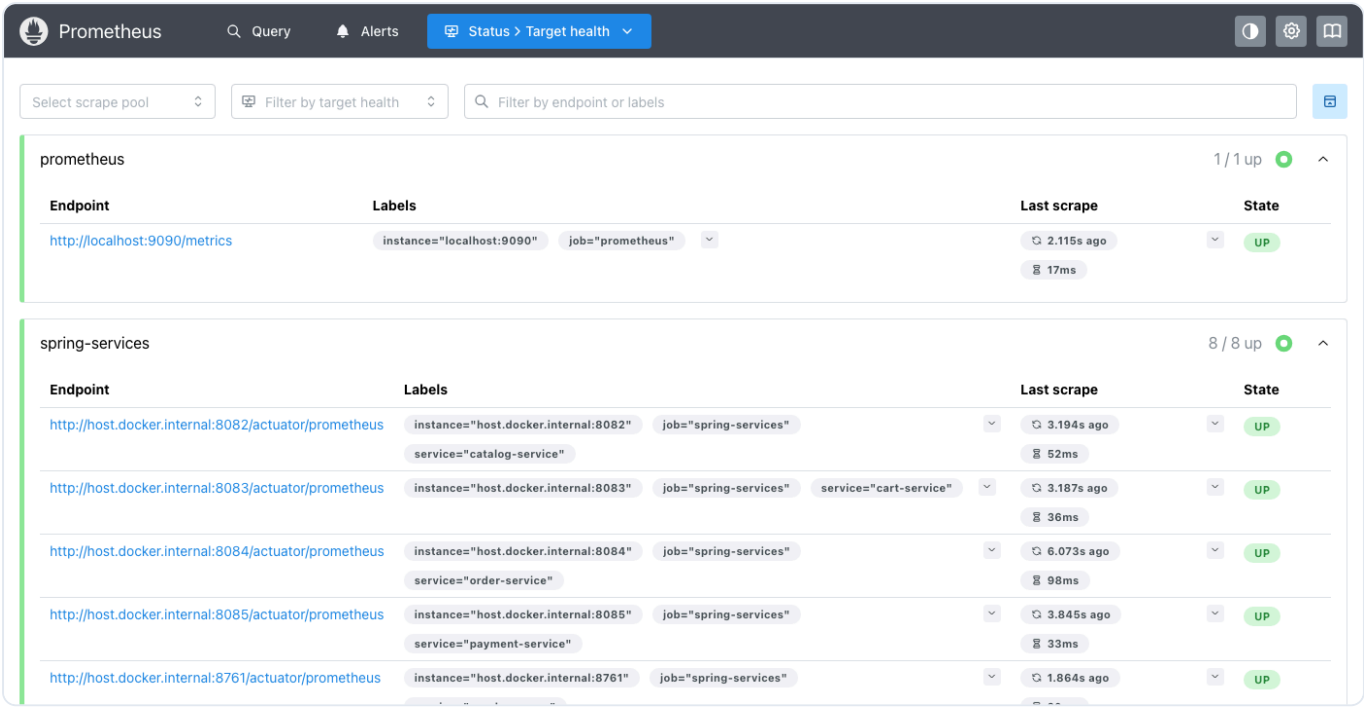
This project deliberately runs the eight Spring services as **host JVMs** (for breakpoints and hot reload) while the supporting infrastructure runs in **containers**. Metrics have to cross that line, and the address depends on which side you are standing on:

```
# Prometheus (in a container) reaching a service (on the host):
  targets: ["host.docker.internal:8084"]     # NOT localhost — that would be
                                             # the container itself

# Grafana (in a container) reaching Prometheus (in a container):
  url: http://prometheus:9090                # Compose service name

# You (on the host) reaching either one:
  http://localhost:9090   http://localhost:3001
```

Three different addresses for the same services, because "localhost" means something different inside each container. The Compose file also sets `extra_hosts: host.docker.internal:host-gateway`, which Docker Desktop provides automatically on macOS and Windows but Linux does not.



Prometheus's own UI at `:9090/targets` — the first place to look when a panel is empty. It shows each target's URL, its labels, the last scrape time, and the exact error when one fails. The `8/8 up` counter is the fastest health check you have.

> **Why each target carries an explicit `service` label.** The `application` tag comes from the service itself, so it vanishes when the service stops. The `service` label is attached by Prometheus at scrape time, so `up{service="order-service"}` keeps working while that service is down — which is precisely when you need it.

8 Try it yourself

With the stack running (`docker compose up -d` , then the eight services):

```
# 1. See exactly what one service publishes — this is the raw input to everything.
curl -s localhost:8084/actuator/prometheus | head -40

# 2. Count how many distinct series a single service exposes (≈270).
curl -s localhost:8084/actuator/prometheus | grep -vc '^#'

# 3. Ask Prometheus which targets it can reach.
curl -s 'localhost:9090/api/v1/query?query=up' | python3 -m json.tool

# 4. Run any PromQL from §5 in the browser, with autocomplete:
#    http://localhost:9090/graph

# 5. Open the dashboard.
#    http://localhost:3001
```

A good first exercise: generate traffic against the storefront, then watch "Request rate" climb about 10–20 seconds later — one scrape interval plus the `rate()` window. That lag is inherent to the pull model, and worth internalising before you write alerts on top of it.

9 Troubleshooting

Symptom	Cause and fix
Target shows 404	The service is running a build without `micrometer-registry-prometheus`, or without `prometheus` in the exposure list. Re-import Maven and restart the service. If it is `eureka-server` or `config-server`, check their own `application.yml` — they never read config-repo (§7).
`unsupported Content-Type "application/json"`	Only config-server does this. The endpoint is not exposed, so Config Server's `/{application}/{profile}` handler answers instead with config data. Expose `prometheus` in its own `application.yml` (§7).
Target shows 401	Usually the same root cause as a 404, wearing a disguise: the endpoint does not exist, Spring forwards to `/error`, and Spring Security challenges *that* path. It resolves once the endpoint really exists — the `/actuator/**` matchers already permit it.
Connection refused	Nothing is listening on that port — usually the service is simply restarting, and Prometheus recovers by itself on the next scrape. If it persists, check the target uses `host.docker.internal` rather than `localhost`, and that the service is actually up.
Panel says "No data"	Distinguish three cases: the target is down (check `/targets`); the metric genuinely has no series yet (an error-rate panel with no errors is correct); or the query references series that do not exist — the histogram-bucket trap in §3 is the classic example.

Dashboard edits vanish on restart	Expected. Provisioned dashboards are re-read from the mounted file. Edit the JSON in `monitoring/grafana/dashboards/`, or export your changes from the UI and save them there.

> **Before this goes anywhere but a laptop:** Grafana here runs with anonymous admin access and no login form, and neither Prometheus nor the actuator endpoints require authentication. That is a deliberate convenience for local development and is not safe to expose.